NETB507 Практика по програмиране

25/26 Autumn Semester
F112088 Sun HeYang
F116120 zhao shengji
F110731 Dmitrii Temnov

Variant 1: ACO shortest-path in a network visualizer

Basic GUI and Generate Graph: (Sun HeYang)

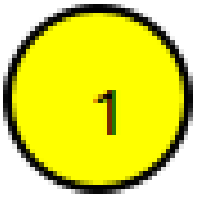
Make a graph needs nodes and edges, I start at nodes:

Nodes:

```h
#ifndef NODEITEM_H
#define NODEITEM_H

#include<QGraphicsEllipseItem>
#include<QGraphicsSceneMouseEvent>
#include<QBrush>
#include<QString>

class NodeItem : public QGraphicsEllipseItem
{
        public:
                NodeItem(const QString& name, const QPointF& pos, qreal radius = 20);

                QString getName() const {return m_name;}
        protected:
                void mousePressEvent(QGraphicsSceneMouseEvent* event) override;
        private:
                QString m_name;
};

#endif
```

```cpp
#include"NodeItem.h"
#include<QPen>
#include<QGraphicsTextItem>

NodeItem::NodeItem(const QString& name, const QPointF& pos, qreal radius) : m_name(name)
{
        setRect(pos.x() -radius, pos.y() -radius, radius *2, radius *2); //
  QGraphicsEllipseItem draw circle as a circle inside of a rectangle, need left upper corner
  point,height,width
        setBrush(QBrush(Qt::yellow));
        setPen(QPen(Qt::black, 2));

        //show Node name
        auto* text = new QGraphicsTextItem(name, this);
        text->setPos(pos.x()-6, pos.y()-10);
}

void NodeItem::mousePressEvent(QGraphicsSceneMouseEvent* event)
{
        setBrush(QBrush(Qt::green));
        QGraphicsEllipseItem::mousePressEvent(event);
}
```

Here are the header and main file of node, node has name and appearance. Also a click event because we need let user choose the start point and end point. Node inherited from <QGraphicsEllipseItem>, Node default radius = 20. setRect function there are top left point, height and width. Because It draw ellipse as a ellipse inside a rectangle. Then the default color I choose yellow, with a black frame. The name of Node should be close to central point of the circle. The mouse press event in protect because this function will be often override in a subclass, and in research people always put function like this in protected. The mousePressEvent for now it make color change from yellow to green. Here is the picture of what node looks like.



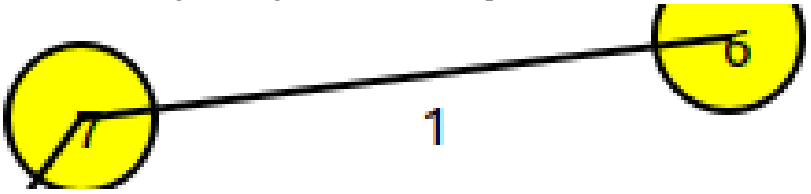
Edges:

Edges should contain two points and weight.

```h
#ifndef EDGEITEM_H
#define EDGEITEM_H

#include<QGraphicsLineItem>
#include<QGraphicsTextItem>

class EdgeItem : public QGraphicsLineItem
{
        public:
                EdgeItem(const QPointF& p1, const QPointF& p2, int weight);
        private:
                QGraphicsTextItem* weightText;
};

#endif
```

```cpp
#include"EdgeItem.h"
#include<QPen>

EdgeItem::EdgeItem(const QPointF& p1, const QPointF& p2, int weight)
{
        setLine(QLineF(p1,p2));
        setPen(QPen(Qt::black, 2));

        QPointF mid = (p1 + p2) / 2;
        weightText = new QGraphicsTextItem(QString::number(weight), this);
        weightText->setPos(mid.x(), mid.y());
}
```

Here I set the line is a black line which width is 2 pixel. And create the weight text at the middle of the edge. A edge should like the picture shown below:



Scene:

We need a scene to show the whole graph, Here is the header and main file.

```h
#ifndef GRAPHSCENE_H
#define GRAPHSCENE_H

#include<QGraphicsScene>
#include"graph.h"
#include"NodeItem.h"
#include"EdgeItem.h"

class GraphScene : public QGraphicsScene
{
        public:
                GraphScene(QObject* parent = nullptr);

                void drawGraph(Graph& graph);

        private:
                QMap<QString, NodeItem*> nodeItems;
};

#endif
```

```cpp
#include"GraphScene.h"
#include<QRandomGenerator>

GraphScene::GraphScene(QObject* parent) : QGraphicsScene(parent)
{
}

void GraphScene::drawGraph(Graph& graph)
{
        clear();
        nodeItems.clear();

        auto vertices = graph.getVertices();

        for (const auto& v: vertices)
        {
                QPoint pos(
                                QRandomGenerator::global()->bounded(100,600),
                                QRandomGenerator::global()->bounded(100,400)
                );

        NodeItem* node = new NodeItem(QString::fromStdString(v), pos);
        addItem(node);

        nodeItems[v.c_str()] = node;
        }

        for (const auto& v1 : vertices)
        {
                for (const auto& v2 : vertices)
                {
                        if(graph.edgeExist(v1,v2))
                        {
                                QPointF p1 = nodeItems[v1.c_str()]->rect().center();
                                QPointF p2 = nodeItems[v2.c_str()]->rect().center();

                                int w = graph.getWeight(v1, v2);
                                addItem(new EdgeItem(p1, p2, w));
                        }
                }
        }
}
```

The constructor is for later in main window connect mainWindow as this graph scene's parent class. Firstly clear all data before, then get vertices and generate random points in this range, save those nodes as pair, nodename and node. This is for easily find those node in creating edges. In create edges, go through all the nodes, if here is an edge connect two nodes, then draw it. Add edge item in the scene with the points at each side and their weight.

MainWindow:

For now the main window should be able to show several button for user interact, which are choose the number of nodes, set wight range of edge, and do regenerate operation. Also because the basic graph class does not has a generate random graph function, so I put that function at here. The Q_OBJECT is for activate the signal and slot, which is for answer user action.

```cpp
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include<QMainWindow>
#include<QGraphicsView>
#include<QSpinBox>
#include<QPushButton>
#include<QHBoxLayout>
#include<QVBoxLayout>
#include"GraphScene.h"
#include"graph.h"

class MainWindow : public QMainWindow
{
        Q_OBJECT

        public:
                MainWindow(QWidget *parent = nullptr);
        private slots:
                void choose8();
                void choose16();
                void regenerate();
        private:
                GraphScene* scene;
                QGraphicsView* view;

                int nodeCount = 8;
                QSpinBox* minWeightBox;
                QSpinBox* maxWeightBox;

                Graph generateRandomGraph();
};

#endif
```

```cpp
#include"MainWindow.h"
#include<QRandomGenerator>
#include<QLabel>

MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent)
{
        scene = new GraphScene(this);
        view = new QGraphicsView(scene);
        view->setRenderHint(QPainter::Antialiasing);

        QPushButton* btn8 = new QPushButton("8 Nodes");
        QPushButton* btn16 = new QPushButton("16 Nodes");
        QPushButton* regenBtn = new QPushButton("Regenerate");

        minWeightBox = new QSpinBox;
        maxWeightBox = new QSpinBox;

        minWeightBox->setRange(1, 100);
        maxWeightBox->setRange(1, 100);
        minWeightBox->setValue(1);
        maxWeightBox->setValue(10);

        QWidget* panel = new QWidget;
        QHBoxLayout* top = new QHBoxLayout;

        top->addWidget(btn8);
        top->addWidget(btn16);
        top->addWidget(new QLabel("Weight range: "));
        top->addWidget(minWeightBox);
        top->addWidget(new QLabel(" - " ));
        top->addWidget(maxWeightBox);
        top->addWidget(regenBtn);

        QVBoxLayout* main = new QVBoxLayout;
        main->addLayout(top);
        main->addWidget(view);

        panel->setLayout(main);
        setCentralWidget(panel);

        connect(btn8, &QPushButton::clicked, this, &MainWindow::choose8);
        connect(btn16, &QPushButton::clicked, this, &MainWindow::choose16);
        connect(regenBtn, &QPushButton::clicked, this, &MainWindow::regenerate);

        regenerate();
}

void MainWindow::choose8()
{
        nodeCount = 8;
        regenerate();
}

void MainWindow::choose16()
{
        nodeCount = 16;
        regenerate();
}

```

First create a new scene and connect to main window as a child class, view->setRenderHint(QPainter::Antialiasing); this is for active anti-aliasing mode. Then make 3 new button for change node number and do regenerate operation. Then create the spin box for set the weight range, and set default value. Make a widget panel for put everything on it, HboxLayout for a horizontal area, I'll put buttons in this area, and a vertical area for combine all other widgets. Connect button with slot functions, and generate a graph when the program start.

```
59
60 void MainWindow::regenerate()
61 {
62         Graph g = generateRandomGraph();
63         scene->drawGraph(g);
64 }
65
66 Graph MainWindow::generateRandomGraph()
67 {
68         Graph g;
69
70         for(int i = 0; i < nodeCount; i++) g.addVertex(QString::number(i).toStdString());
71
72         int minW = minWeightBox->value();
73         int maxW = maxWeightBox->value();
74
75         int maxEdges = nodeCount * (nodeCount - 1)/2;
76         int edgeCount = QRandomGenerator::global()->bounded(maxEdges/3, maxEdges*2/3);
77
78         for (int i = 0; i < edgeCount; i++)
79         {
80                 int u = QRandomGenerator::global()->bounded(nodeCount);
81                 int v = QRandomGenerator::global()->bounded(nodeCount);
82
83                 if(u == v) continue;
84
85                 if(g.edgeExist(QString::number(u).toStdString(), QString::number(v).toStdString()) || (g
86                 {
87                         continue;
88                 }
89
90                 int weight = QRandomGenerator::global()->bounded(minW, maxW + 1); //bounded is [a, b) ,
91
92                 g.addEdge(QString::number(u).toStdString(), QString::number(v).toStdString(), weight);
93
94         }
95
96         return g;
97 }
```

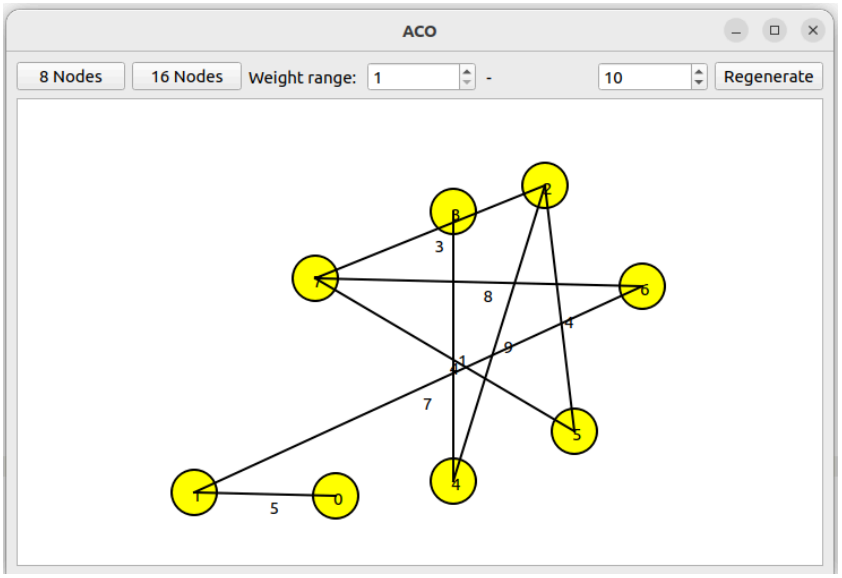
Here is the main function for generate a graph, set max edges, and random generate a number from max edges. The do generate loop, random choose two point in the whole graph and try connect them if they haven't got a edge. Then random set a weight, add edge in to graph, and return the graph.

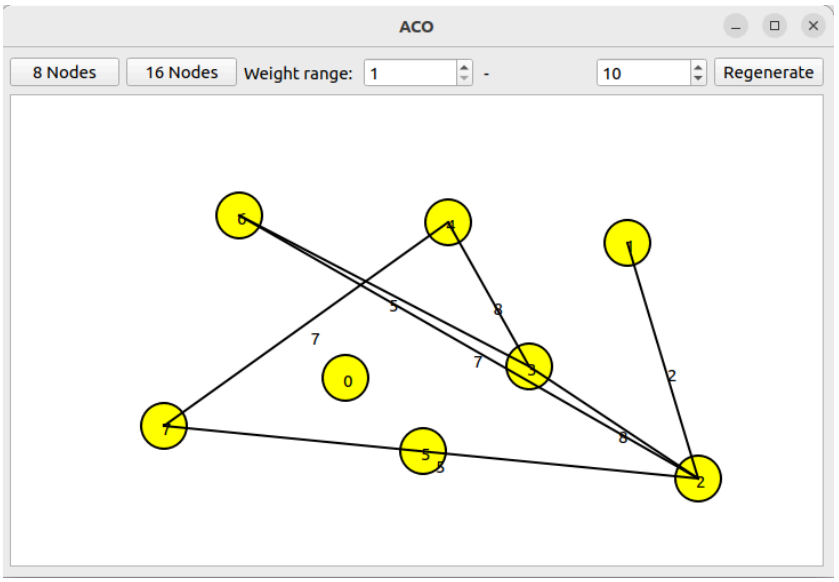
CMAKE:

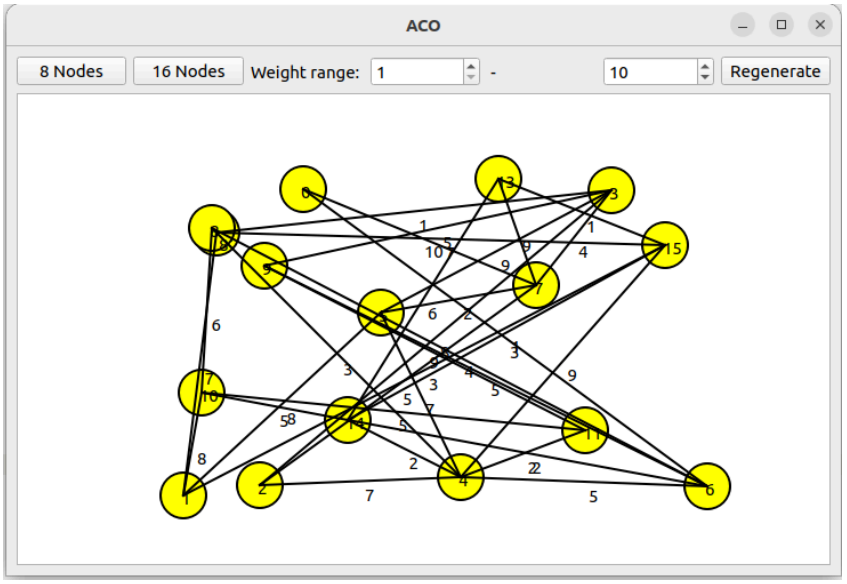
For now to test the whole program I used cmake, add all files in it and out put ACO.exe

```
1 cmake_minimum_required(VERSION 3.16)
2 project(ACO)
3
4 set(CMAKE_CXX_STANDARD 17)
5 set(CMAKE_AUTOMOC ON)
6
7 find_package(Qt6 REQUIRED COMPONENTS Widgets)
8
9 set(SOURCES
10         main.cpp
11         MainWindow.cpp
12         GraphScene.cpp
13         graph.cpp
14         NodeItem.cpp
15         EdgeItem.cpp
16 )
17
18 set(HEADERS
19         MainWindow.h
20         GraphScene.h
21         graph.h
22         NodeItem.h
23         EdgeItem.h
24 )
25
26 qt_add_executable(ACO
27         ${SOURCES}
28         ${HEADERS}
29 )
30
31 target_link_libraries(ACO
32         PRIVATE Qt6::Widgets
33 )
```

Here are some example of the program for now:







Zhao Connect the ACO algorithm with a graphical user interface and add visualization functionality.

```cpp
AcoController::AcoController(SelectionController* selector, QWidget* parent)
    : QObject(parent), m_selector(selector), m_parentWidget(parent)
{
    // Create widgets
    m_runBtn = new QPushButton("Run ACO", parent);
    m_resetBtn = new QPushButton("Reset", parent);
    m_infoLabel = new QLabel(parent);
    // stop use until selection two node
    m_runBtn->setEnabled(false);
    m_infoLabel->setText("Selected: (none)");

    connect(m_runBtn, &QPushButton::clicked, this, &AcoController::onRunClicked);
    connect(m_resetBtn, &QPushButton::clicked, this, &AcoController::onResetClicked);
}

void AcoController::addToLayout(QHBoxLayout* layout)
{
    layout->addWidget(m_runBtn);
    layout->addWidget(m_resetBtn);
    layout->addWidget(m_infoLabel);
}

QString AcoController::selectionText() const
{
    if (!m_selector) return "Selected: (none)";
    if (!m_selector->hasSource())
        return "Selected: click source";
    if (!m_selector->hasDestination())
        return "Selected: " + m_selector->source() + " -> (click destination)";

    return "Selected: " + m_selector->source() + " -> " + m_selector->destination();
}
```

```cpp
void AcoController::sync()
{
    if (!m_selector) return;
    m_runBtn->setEnabled(m_selector->ready());// Run only when ready
    m_infoLabel->setText(selectionText());// Update small label
}

void AcoController::onResetClicked()
{
    if (m_selector)
    {
        m_selector->clear();
    }
        sync();

}

void AcoController::onRunClicked()
{
    if (!m_selector) return;
    if (!m_selector->ready())
    {
        QMessageBox::information(m_parentWidget, "ACO", "Please select source and destination first.");
        sync();
        return;
    }

    emit runRequested(m_selector->source(), m_selector->destination());


    sync();
}
```

ACOController.cpp

the main functions are to manage button states and tooltip text, whether a button is active, and to pass the "run algorithm request" to the MainWindow via a signal,

The most important codde is :

```cpp
emit runRequested(m_selector->source(), m_selector->destination());
```

It signals to MainWindow the user clicked 'Run,' and the start and end points are these two. run the algorithm and update the interface.

```cpp
#include"SelectionController.h"
#include"GraphScene.h"
#include <QColor>

SelectionController::SelectionController(GraphScene* scene): m_scene(scene)
{

}

void SelectionController::setScene(GraphScene* scene)
{
    m_scene = scene;
    clear();

}

void SelectionController::clear()
{
    m_source.clear();
    m_dest.clear();
    applyColors();
}
```

```cpp
void SelectionController::handleNodeClicked(const QString& nodeId)
{
    if(m_source.isEmpty())
    {
        m_source = nodeId;// dangzuo
        m_dest.clear();
        applyColors();
        return;

    }
      if (m_dest.isEmpty())
    {
        if (nodeId == m_source) return; // same no
        m_dest = nodeId;
        applyColors();
        return;
    }

       m_source= nodeId;//have both 2 node reset
       m_dest.clear();
       applyColors();
}
void SelectionController::applyColors()
{
  if (!m_scene) return;
  m_scene->resetStyles();
  if (!m_source.isEmpty())//set color
  m_scene->setNodeColor(m_source, QColor(Qt::blue));

  if (!m_dest.isEmpty())
  m_scene->setNodeColor(m_dest, QColor(Qt::red));
```

SelectionController.cpp

This code allows to clear the scene and its state, including clearing the source and destination, and refreshing the default color.It can use if statements to determine the nodes and color them.

```cpp
AcoResult AcoRunner::run(Graph& graph,const std::string& source,const std::string& destination)

{
 AcoResult result;
 std::map<std::string, std::map<std::string, int>> adjacency;

for (const auto& u : graph.getVertices())
 {
for (const auto& v : graph.getVertices())
    {
if (graph.edgeExist(u, v))
  {
    adjacency[u][v] = graph.getWeight(u, v);
    adjacency[v][u] = graph.getWeight(u, v);
}
 }
}
  const int maxAttempts = 30;
for (int attempt = 0; attempt < maxAttempts; ++attempt)
{
    ACOService aco(adjacency);
    result.path = aco.findPath(source, destination);

  if (!result.path.empty())
      break;
}
  result.totalWeight = calculateTotalWeight(graph, result.path);
  return result;
}
```

```cpp
int AcoRunner::calculateTotalWeight(Graph& graph,const std::vector<std::string>& path)
{
   int sum = 0;
   if (path.size() < 2)
   return 0;
  for (size_t i = 0; i + 1 < path.size(); ++i)
    {
    const std::string& u = path[i];
    const std::string& v = path[i + 1];

   if (graph.edgeExist(u, v))
    sum += graph.getWeight(u, v);
    else if (graph.edgeExist(v, u))
    sum += graph.getWeight(v, u);


    }
return sum;
}
```

ACOrunner.cpp

The run() function converts the Graph into an adjacency and attempts to call `ACOService::findPath` a maximum of 30 times to obtain the path. After obtaining the path, it uses `calculateTotalWeight` to accumulate the total Weight according to the edge weights of the Graph and returns it to the UI for highlighting and display.

```cpp
void GraphScene::setNodeClickHandler(const std::function<void(const QString&)>& handler)
{
    m_nodeClickHandler = handler;//The click logic is saved to the GraphScene member variable.
}//zhao

void GraphScene::resetStyles()
{
    for (auto it = nodeItems.begin(); it != nodeItems.end(); ++it)
    {
        if (it.value()) it.value()->setColor(Qt::yellow);// if empty pass or change to yellow
    }
}//zhao

void GraphScene::setNodeColor(const QString& nodeId, const QColor& color)
{
    if (!nodeItems.contains(nodeId))
    return;                             //check if ondeitems have node id,OR
    nodeItems[nodeId]->setColor(color);
}//zhao

void GraphScene::highlightPath(const std::vector<std::string>& path)
{
    resetStyles();
     for (const auto& id : path)
    {
        QString qid = QString::fromStdString(id);
        setNodeColor(qid, Qt::green);
    }
}//zhao
```

Graphscene.cpp

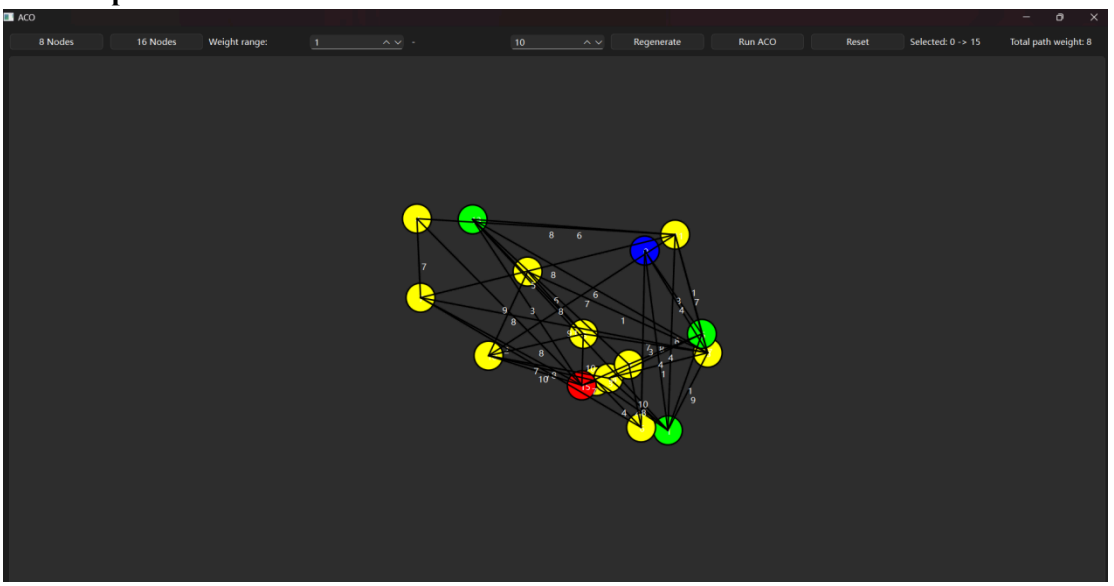
This code belongs to the core logic of GraphScene's display and interaction layer: it registers externally provided node click callbacks through setNodeClickHandler, so that GraphScene is not concerned with specific business logic; it manages the color state of nodes uniformly through resetStyles and setNodeColor, ensuring that the old style is cleared before each selection or highlight; and it highlights the nodes on the path uniformly (in green) after the algorithm returns the path through highlightPath, thereby decoupling the "algorithm result" and "graphic display" and ensuring that the interface state is clear and controllable.

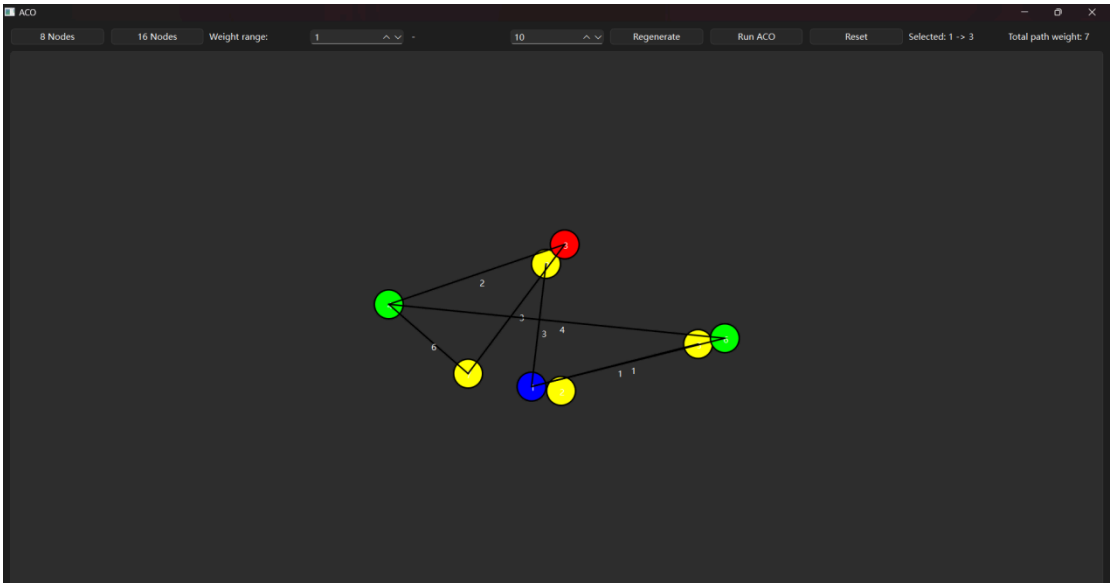
```cpp
scene->setNodeClickHandler([this](const QString& nodeId)
{
 this->onNodeClicked(nodeId);
});//zhao set a return when click node weill return back mw

connect(aco, &AcoController::runRequested,this, [this](const QString& src, const QString& dst)
{
 AcoRunner runner;
 AcoResult r = runner.run(
 currentGraph,
 src.toStdString(),
 dst.toStdString()
 );
 if (r.path.empty())
 {
     scene->setNodeColor(src, Qt::blue);//even no pat color it
     scene->setNodeColor(dst, Qt::red);
     resultLabel->setText("Total path weight: No path found (graph direction/connectivity issue).");
     return;
 }
 scene->highlightPath(r.path);
 scene->setNodeColor(src, Qt::blue);
 scene->setNodeColor(dst, Qt::red);//make sure no be cover
 resultLabel->setText(QString("Total path weight: %1").arg(r.totalWeight));
;//zhao
```

Mainwindow.cpp

This code pushes the node clicks in the graph layer up to MainWindow via callback to maintain the source/destination selection state. When the user clicks Run, AcoController sends a runRequested signal, and MainWindow calls AcoRunner to run ACO on the currentGraph. If there is no path, it provides a prompt and retains the start and end point colors. If there is a path, it highlights the path and overwrites the start point blue/end point red again. Finally, it updates the total weight display, achieving decoupling between the algorithm and the UI and stable state-driven visualization.

This is Operation effect





Algorithm Overview

The Ant Colony Optimization algorithm is a metaheuristic shortest path search algorithm inspired by the foraging behavior of ants. Ants deposit pheromones on paths they traverse, and subsequent ants are more likely to follow paths with stronger pheromone trails. Over time, this leads to the discovery of optimal paths.

Implementation Details

Core Components

1. ACOService (`ACOService.h`, `ACOService.cpp`)

The main class implements the ACO algorithm. It manages:

- **Graph representation: Uses the `Graph` class to represent the weighted graph**
- **Pheromone matrix: Maintains pheromone levels for each edge in the graph**
- **ACO parameters: Configurable parameters that control algorithm behavior**

2. Graph Structure (`graph.h`, `graph.cpp`)

Represents the graph as an adjacency matrix using nested maps:

map<string, map<string, int>> adjMatrix

- **Vertices are identified by string names**
- **Edge weights are stored as integers**
- **Supports bidirectional edges**

3. AcoRunner (`AcoRunner.h`, `AcoRunner.cpp`)

Wrapper class that:

- **Converts the `Graph` structure to an adjacency matrix format**
- **Executes the ACO algorithm**
- **Handles multiple attempts if no path is found initially**
- **Calculates the total weight of the resulting path**

ACO Parameters

The algorithm uses the following default parameters:

Parameter	Value	Description
`alpha`	1.0	Pheromone importance factor - controls how much ants rely on existing pheromone trails
`beta`	2.0	Heuristic importance factor - controls how much ants prefer shorter edges
`evaporationRate`	0.4	Rate at which pheromones evaporate (40% per iteration)
`depositAmount`	100.0	Base amount of pheromone deposited by ants
`antsNumber`	N	Number of ants equals the number of vertices in the graph
`maxIterations`	100	Maximum steps an ant can take before stopping
`pheromoneIterations`	30	Number of outer iteration rounds

Algorithm Flow

The `findPath(source, destination)` method implements the following steps:

1. Initialization
// Initialize pheromone matrix with value 1.0 for all edges
pheromoneMatrix[from][to] = 1.0;

2. Main Loop (30 iterations)

For each iteration:

a. Ant Creation

- **Create `N` ants (where `N` = number of vertices)**
- **All ants start at the source vertex**

b. Ant Movement

Each ant moves until it reaches the destination or exceeds `maxIterations`:

- **Current position: Get the ant's current vertex**
- **Next vertex selection: Use probabilistic selection based on transition probability**

Transition probability calculation:

$$P(i,j) = (\tau_i^{\alpha} \times \eta_i^{\beta}) / \Sigma(\tau_i^{\alpha} \times \eta_i^{\beta})$$

Where:

  - **τ_i = pheromone level on edge (i,j)**
  - **η_i = 1/weight(i,j) (heuristic value - shorter edges are more attractive)**
  - **α = pheromone importance (1.0)**
  - **β = heuristic importance (2.0)**
  - **The sum is over all available unvisited neighbors**
- **Roulette wheel selection: Randomly select next vertex based on calculated probabilities**
- **If an ant gets stuck (no available unvisited neighbors), it resets to the source vertex**

c. Pheromone Update

After all ants complete their paths:

Deposit: Ants that successfully reached the destination deposit pheromones:

deposit = depositAmount / pathLength

This prevents the algorithm from getting stuck in local optima and allows exploration of new paths.

3. Best Path Selection
After all the iterations are done, return the shortest path found among all successful ant paths. Shorter paths receive more pheromone per edge, making them more attractive in

future iterations.

Evaporation: All edges lose pheromone:

new_pheromone = old_pheromone × (1 - evaporationRate)